

Distributed Hash Table and the Chord protocol

Assignment 1B - INF-3200 Distributed Systems Fundamentals

Sindre Frantzen Dalvik

UiT

Tromsø, Norway

sda108@uit.no

Jens Christian Valen Leynse

UiT

Tromsø, Norway

jeley4465@uit.no

Sivert Bottolfsen

UiT

Tromsø, Norway

sbo101@uit.no

Abstract—This implementation is a distributed peer-to-peer hash table based on the Chord paper, designed to run on the UiT cluster. The application is a file storage system that supports GET, POST, and /network operations. The files are relatively equally spread among the nodes in the cluster. After initializing the system, it does not support the leaving and joining of nodes.

Index Terms—Distributed scalable algorithms, peer-to-peer networks, File system

I. INTRODUCTION

In this assignment, we implemented a Distributed Hash Table (DHT) based on the Chord protocol, focusing on key-value storage and retrieval across multiple network nodes. Chord uses consistent hashing to efficiently map nodes and keys, enabling correct insertion and retrieval of data while navigating the ring structure with finger tables.

We also evaluated the system's performance by measuring throughput across different network sizes, analyzing how the number of nodes impacts the efficiency of PUT and GET operations, and examining the trade-offs in scalability and fault tolerance.

II. DESIGN

The system is designed to implement a distributed hash table (DHT) using the Chord protocol [1]. Each node in the network is responsible for a specific segment of the key space, allowing for scalable, fault-tolerant data storage and retrieval.

Key elements of the design include:

- **Node Initialization:** Each node is initialized with a unique identifier determined by hashing its IP address mod 2^{16} . A node keeps track of its predecessor and successor in the ring, which helps in data lookups and key storage.
- **Finger Table:** The finger table is a routing table maintained by each node to optimize search operations. Instead of just knowing the immediate successor, nodes store information about nodes further along in the ring, improving lookup efficiency.
- **Key Storage:** Keys are stored at the node whose identifier matches the hash of the key. A node is responsible for all keys that fall between its predecessor and itself.
- **Request Handling:** The system uses an HTTP server at each node to handle requests for storing and retrieving keys. Nodes collaborate to find the appropriate location

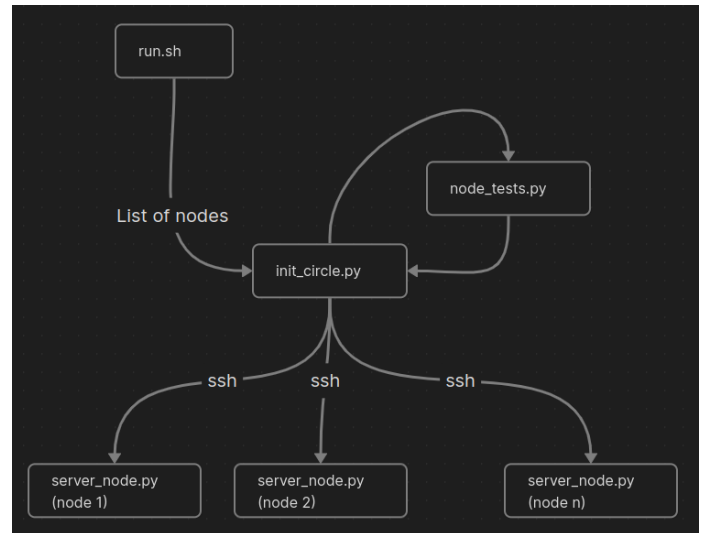


Fig. 1. Data Flow Diagram

for each key by forwarding requests to other nodes if necessary. Communication between nodes is done using HTTP requests.

- **Consistent Hashing:** The Chord protocol is based on consistent hashing, which allows for the dynamic addition and removal of nodes without significant reorganization of the network. Keys are only moved between adjacent nodes in the ring when nodes join or leave the network.
- **Timeout Mechanism:** Each node is designed with a time-to-live mechanism that automatically terminates the server after a given time. This is to ensure excessive resource usage.

Advantages:

- **Scalability:** Chord is scalable as the lookup time grows logarithmically with the number of nodes, making it suitable for large distributed systems.
- **Fault Tolerance:** By distributing data across multiple nodes, Chord enhances fault tolerance. Even if some nodes fail, the system can still function, with data replicated or redistributed among other nodes.
- **Load Balancing:** Consistent hashing ensures that keys are distributed uniformly across the ring, which helps balance the load on individual nodes.

Disadvantages:

- **Join/Leave Overhead:** Chord inherently incurs overhead when nodes join or leave. The system must reconfigure finger tables and redistribute key ranges, which can affect performance.
- **Latency:** Network communication between nodes introduces latency, which may impact the time required to handle GET and PUT requests, especially as the system scales.

III. IMPLEMENTATION

Node Initialization: Each node begins by calculating its unique identifier using a hashing function. This identifier is used to determine the node's position in the ring. Predecessor and successor nodes are assigned based on this position, forming the consistent hash ring structure. The finger table is created during node initialization.

The values for the nodes index, successor and predecessor are tested before deploying to the cluster. This is to ensure that no node index collisions occur. And to ensure that the chord loop is correctly iterable in a ring, without any duplicate nodes, or missing nodes. If all tests pass, the nodes are deployed in the cluster.

Request Routing: Nodes handle client requests for key-value storage (PUT) and retrieval (GET) by either fulfilling the request locally or forwarding it to the appropriate node in the ring. If a node receives a key request outside its range, it uses the "find successor" logic to forward the request based on its finger table.

Key Storage and Retrieval: When storing a key, the node checks whether the key falls within its responsibility range. If so, the key is stored locally; otherwise, the request is forwarded to the correct node. The retrieval process works similarly, with nodes either returning the value or forwarding the request.

Efficient Lookup with Finger Tables: The "find successor" algorithm utilizes the finger table to skip intermediate nodes, allowing the system to locate a node in logarithmic time relative to the total number of nodes. This minimizes the number of hops a request needs to traverse.

HTTP Server for Communication: Each node runs a lightweight HTTP server to handle incoming requests from both clients and other nodes. The server processes requests for storing and retrieving keys, updating node information, and resolving successor queries.

Node Deployment and Integration: The system supports the deployment of multiple nodes, which are assigned random ports and initialized with their respective finger tables. Nodes are deployed by executing a bash command via SSH, ran from a Python program. The bash command calls are multi-threaded, to avoid the bottleneck of sequentially waiting for each node to complete deploying. Once deployed, nodes automatically join the ring and begin participating in the DHT.

Debugging and Logging: The system includes functionality for writing debug logs during node operations, such as key lookups, finger table construction, and HTTP request handling.

This helps trace the flow of requests and identify issues during node communication or key assignment.

IV. EXPERIMENT

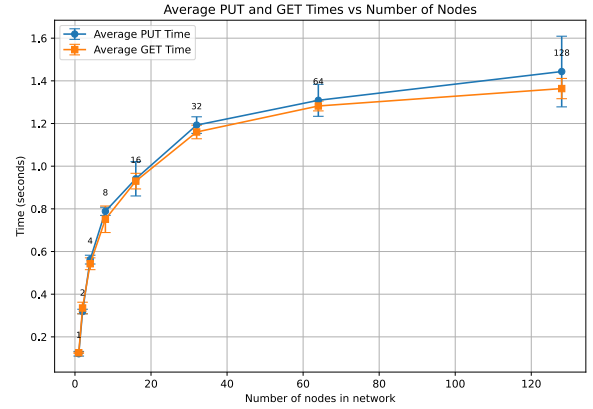


Fig. 2. Average PUT and GET times of 100 different values in DHT

Figure 2 shows that the algorithm has a logarithmic time complexity which agrees with the chord paper [1].

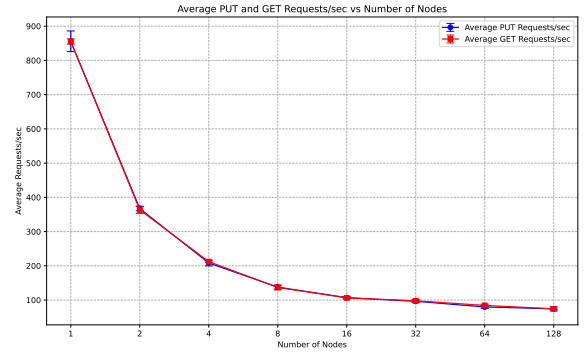


Fig. 3. Throughput (PUTs and GETs per second)

Figure 3 shows the throughput of the system. The experiment did 5 runs and averaged them to calculate the time and standard deviations.

The graph in figure 4 shows that the finger table implementation scales much better than the linear implementation which has a very low throughput with many nodes.

V. DISCUSSION

Simulation and tests before deployment:

As mentioned in the implementation segment, the nodes gets their index, predecessor, and successor values, before they are deployed in the cluster. These values are used in tests that automatically run on deployment. If the tests do not pass, the application stops, and no nodes are deployed. A disadvantage of this is that it results in a lot of extra code, that does not directly contribute to the functionality

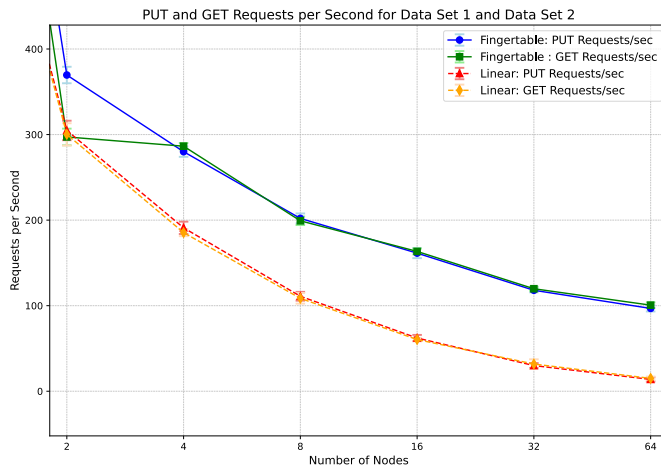


Fig. 4. Comparison between fingertable and linearly without fingertable (PUTs and GETs per second)

of the application. An advantage of this, is that potential errors originating from nodes with non-sensible values are caught early. Also it reduces the load on the cluster when debugging, especially when running with a high number of nodes. Although it slightly increases the load on the setup node that runs the tests. So running from the front-end node is not recommended. We did however implement a check that prevents multi threading of the setup if the program is ran from the front-end node, just in case. Although this results in the deployment taking much longer (if ran from the front-end node).

Use of the cluster file system:

When debugging the servers, we used the file-system to write debug prints to a file in the cluster. The assignment stated that use of the file system for communication between the nodes is not allowed. In our implementation, nodes write to a shared file "debug.log", in order to debug. They do not use the file system to communicate with each other. It is only used for debugging. If we removed the file, and all debug prints, it would not affect the functionality of the program. Another use of the file-system is the "node.txt". This file is used for convenience, as it allows running both the implementation and the chord tester at the same time. The implementations functionality would be the same if "node.txt" is not used, only difference being that one would have to manually copy-paste the name of a node into the chord tester. So to be clear: In no way do we use the file system to communicate.

Implementation compared to the Chord paper:

We followed the Chord paper quite closely. We used hashing to decide where on the ring the nodes and files are placed. We also closely followed the pseudo-code suggested by the paper. Except for the leaving and joining of nodes. The finger-table deviates a little from the pseudo code. In our code, the node first checks if the item is in the current node, before looking in the successor / finger-table. The Chord paper on the other

hand always checks if the successor has it. This was because the algorithm will jump to the node that contains the key if found in the finger table. The node that has the key needs to know if itself has the key in addition to if the successor has it.

Consistent hashing and Chord

Before implementing Chord, and a finger-table, we had a consistent hashing, neighbour search. That iterated over each neighbour like a linked list. This endpoint can still be found in the code, and it can be used by changing the config file.

Hash collision of nodes:

Early on we had a bug where two or more nodes hashed to the same index. We improved on this by expanding the Chord circle size to 2^{16} , and using the SHA-1 hashing algorithm. After that, we have not observed hash collisions happening. It is still theoretically possible, but extremely unlikely,

Hash collision of files:

In the implementation, only one file is allowed per id. If a put happens to a index already containing a file, that file is overwritten even if it was different keys such as "dog" and "cat". A potential improvement on this could be to have a hash-map inside each node, with a different hash only for the files. In python we would do this using a dictionary, where the file name is the key. That would allow files of different names, that hashes to the same index in the Chord to be stored at the same time.

Redundancy and fault tolerance:

We have not implemented redundancy. This could be done by having each file stored in at least two nodes. If we had support for leaving of nodes (fault tolerance), and redundancy, the files stored in the system would be much safer. In the current implementation, the system stops working if a node fails. And all the files from the node is lost.

VI. CONCLUSION

There are no bugs that we are aware of, except node hash collision, although very rare. The peer-to-peer file system works as intended for all operations and tests provided in the assignment. And uses the Chord system for finding the correct node. However we could have taken the implementation a bit further by allowing more files per index. And potentially joining and leaving of nodes.

REFERENCES

- [1] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup protocol for internet applications," *IEEE/ACM Transactions on Networking*, vol. 11, no. 1, pp. 17–32, 2003.
- [2] M. v. Steen and A. S. Tanenbaum, *Distributed Systems: Principles and Paradigms*, 4th ed., Pearson Education, 2023.